

BASIC SYNTAX

Syntax in Python is a set of rules that determine how program commands are correctly written and organized. Its importance lies in enabling the **interpreter** to understand and execute the code. (*An interpreter is a program that **reads and executes the code line by line** without first translating it into machine language.*). Correct syntax ensures that the program works as intended and facilitates code maintenance and development. Without it, Python simply does not “know” what to do.

"Computers understand their own language best. It is easier for us to understand them than for them to understand us."

Example 1.1:

In this example, we want to write a simple program that responds to the number entered by the user. First, the number is multiplied by 2. If the number is greater than 5, the program outputs the message "The number is greater than 5"; otherwise, it outputs the message "The number is less than 5."

Before we start writing the actual code in Python, it makes sense to first write **pseudocode**—a simple description of the program steps in natural language. Pseudocode allows us to clearly see the logic of the problem and more easily plan how the program should work.

The pseudocode for this program would be:

Start

Prompt the user for a number

Multiply number by 2

If number > 5

Output "Number is greater than 5"

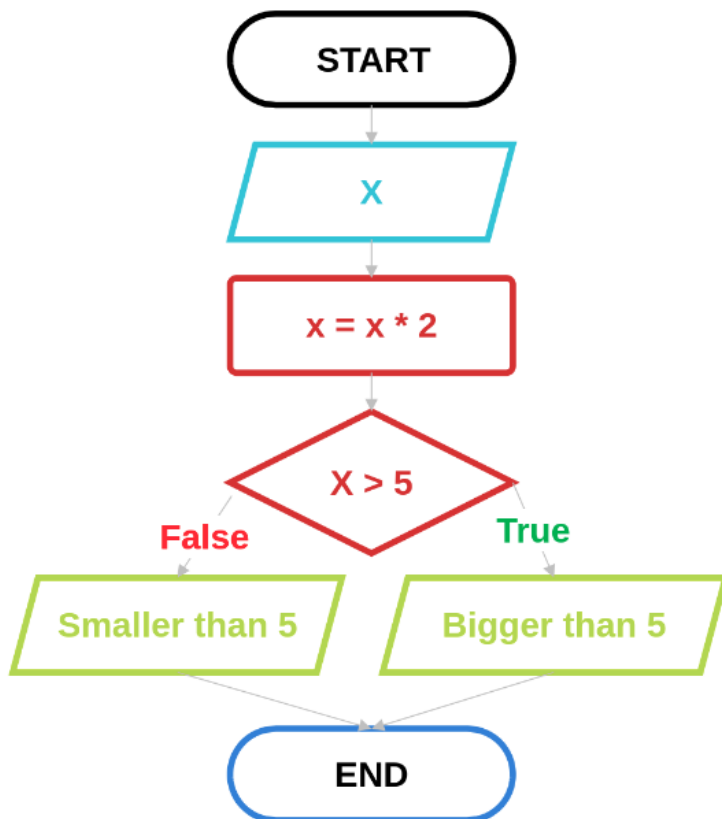
Otherwise

Output "Number is less than 5"

End

We can then create a **flowchart**, a graphical representation of the algorithm that shows the sequence of steps and decisions in the program. The flowchart helps us to **visually understand** the program flow and see how different decisions affect its execution.

The flowchart for this program would be:



Flowchart notation:



START/END



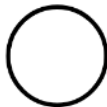
OPERATION



DECISION



INPUT/OUTPUT



BREAK



CONTINUE

Once we have pseudocode and a flowchart, we move on to writing the actual **Python code**, where we will clearly see how the logic is expressed in the syntax of the language and how the computer interprets our commands.

REMEMBER :

The lexicon represents the basic building blocks of the language, i.e., terms and symbols that the computer can recognize and distinguish, while the syntax determines how these elements must be put together so that the commands make sense and are accepted without errors. Only when we write a correct expression does semantics come into play, determining the meaning of that expression, i.e., what exactly happens when the code is executed. Therefore, entering a number has no meaning unless it is used in a calculation, and the output of the text remains just a normal display on the screen unless we connect it to the logic of the program that gives it a purpose.

The Python code for this program would be:

```
x = int(input("Enter a number: "))
x = x * 2
if x > 5:
    print("X is greater than 5")
else:
    print("X is less than 5")
exit()
```

Conclusion:

In this example, we went through three important steps in program development. First, we wrote pseudocode to describe the logic in a simple way, then we created a flowchart to visually represent the execution sequence, and finally we converted everything into real Python code. This way, you can see how an idea from natural language (pseudocode) and visual representation (flowchart) is transformed into concrete commands that the computer understands.

At this stage, we have not yet delved deeply into all the rules of code writing, as we will later deal with the detailed **syntax of Python** individually for each element using examples and explanations. Right now, the most important thing is to understand the logic and flow of the program, while the technical rules of writing, such as the correct use of indentation, characters, and keywords, will be covered step by step in the following chapters...

Comments

Comments in Python are used to explain and mark parts of the code to make it easier for ourselves or others to understand the program. Python marks them with the # character (for single-line comments) or with triple quotation marks `"""` or `'''` (for multi-line comments).

```
# This is a comment!  
""" I'm also a  
comment! """
```

It is important to understand that the Python interpreter completely **ignores comments** when executing the program. They have no effect on the execution of the code and do not change the results. Their purpose is solely to communicate with the programmer, not with the computer.

We must take them into account because they are crucial for writing **readable and maintainable code**.

Without comments, more complex programs become difficult to understand and later difficult to change, even for the programmer who wrote them. A well-written comment helps to clarify the logic of the program, facilitates debugging, and speeds up teamwork.

Back to **Example 1.1:** Our Python program with comments would look like this:

```
# Get the value of x from the user
x = int(input("Enter a number: "))
# Multiply the number by two
x = x * 2
if x > 5:
    """If x is greater than 5, print out "X is greater than 5"""
    print("X is greater than 5")
else:
    """If x is smaller than 5, print out "X is smaller than 5"""
    print("X is less than 5")
# Exit the program
exit()
```

Variables

Variables are a **fundamental concept** in almost every programming language, including Python. They are used to store data in the computer's memory so that we can use, modify, and process it while the program is running. In Python, a variable is actually a name (identifier) that “points” to specific data. When we create a variable, we don't have to specify its type in advance. Python automatically determines the data type based on the value we assign to it (dynamically typed language).

Example 1.2:

```
x = 5
print(x) # 5
x = "Hello"
print(x) # Hello
x = 3.14
print(x) # 3.14
```

When we write `x = 5`, Python creates an object of type integer and the name `x` “refers” to it. When we then write `x = “Hello”`, Python no longer associates `x` with a number, but with a new object of type string. The same thing happens when we later assign a decimal value; `x` now refers to an object of type float. It is important to understand that Python does not “define” the type of a variable. It can change during the execution of the program, which offers great flexibility but also requires caution to avoid errors when mixing incompatible data types (more on this later...).

The basic properties of variables in Python are: Dynamic typing: The data type is automatically determined when a value is assigned. There is no declaration; the variable is created the moment a value is first assigned to it. Variable names must be clear and descriptive and comply with naming conventions: They may contain words, numbers, and underscores (`_`), may not begin with a number, and may not be **Python reserved words**:

(False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield).

Example 1.3: Valid and invalid names

```
name = "Tom" # valid
num1 = 100 # valid
_price = 9.99 # valid
1num = 100 # invalid (starts with number)
for = 5 # invalid (reserved word)
```

In Python, values are assigned using the equal sign (=). For example, when we write `x = 5`, we are saying that the name of the variable `x` refers to the value 5, which is stored in memory. The left side of the expression always represents the name of the variable, while the right side represents the value or expression whose result is assigned. The value can be a number, text, the result of a calculation, or even another variable.

It is also possible to assign multiple values, even the same name to multiple variables. This means that we can assign one or more values to multiple variables in a single line, and Python will link them according to **the assignment rules**.

Example 1.4:

```
x = y = z = 0
print(x, y, z) # Prints: 0 0 0
a, b, c = 1, 2, 3
print(a, b, c) # Prints: 1 2 3
x = x = 5
print(x) # Prints: 5
```

QUESTION: What does the print function output at the end of the following program?

```
x, y = 5, 10
x, y = y, x
print(x, y)
```

In Python, `del` is used to **delete variables**. When we delete a variable with `del`, it no longer exists in memory, so any further use of that name will cause an error.

Example 1.5:

```
x = 10
del x
print(x) # Error message
```

Types of variables by scope

In Python, variables are also distinguished by **scope**, i.e., the part of the program in which they are available and can be used. There are two basic types:

1) **Global variables** are defined outside of all functions and are available throughout the entire program. All functions can read the values of global variables and, in some cases, also change them by using the keyword `global`.

Example 1.5:

```
x = 10 # global variable
def show():
    print(x) # the function can use the global variable
show() # prints: 10
```

2) **Local variables** are defined within a function and only exist while the function is running. Once the function ends, local variables cease to exist. (We will examine variables in functions and their scope in more detail later when we discuss functions and their behavior.)

Example 1.7:

```
def fun():
    y = 5 # local variables
    print(y)
fun() # prints: 5
# print(y) #Error(y is out of scope)
```

In Python, there are no true constants as in some other languages (e.g., C or Java). This means that Python does not prevent the value of a variable that we want to treat as a constant from being changed. However, according to programming convention, values that must not be changed are marked with **capital letters** (PI=3.14169, E=2.71828).